

IBM Hardware - VQC Iris & Breast Cancer

Modell: VQC (RealAmplitudes)

Backends: ibm_marrakesh (oder ibm_fez, ibm_kingston)

Datensätze: Iris (3 Klassen) + Breast Cancer (2 Klassen)

Referenz: 02_iris_fair.ipynb + 04_breast_cancer_fair.ipynb

⚠ Vor dem Ausführen:

- Kontingent prüfen (Zelle 1)
- Erst Iris, dann Breast Cancer
- Jobs werden asynchron abgeschickt - Ergebnisse per Job-ID abrufen
- SVM nur ausführen wenn noch ausreichend Kontingent vorhanden (Zelle 10)

0. Imports

```
In [ ]: import sys, os
        sys.path.append(os.path.abspath(".."))

import numpy as np
import pandas as pd
import time
import warnings
warnings.filterwarnings("ignore")

from sklearn.datasets import load_iris, load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.decomposition import PCA
from sklearn.metrics import accuracy_score, f1_score, classification_report
from sklearn.svm import SVC

from qiskit.circuit.library import zz_feature_map, real_amplitudes
from qiskit_machine_learning.algorithms import VQC
from qiskit_machine_learning.kernels import FidelityQuantumKernel
from qiskit_machine_learning.optimizers import SPSA
from qiskit_ibm_runtime import QiskitRuntimeService, SamplerV2 as Sampler
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager

from utils import save_result

print("Imports OK")
```

1. IBM Verbindung & Kontingent prüfen

⚠ Immer zuerst ausführen - vor jedem Job-Block.

```
In [ ]: service = QiskitRuntimeService()

# Backend wählen
backend = service.backend("ibm_marrakesh")
print(f"Backend: {backend.name}")
print(f"Qubits: {backend.num_qubits}")
print(f"Status: {backend.status().status_msg}")
print()

# Kontingent
usage = service.usage()
print(f"Verbrauchte QPU-Zeit: {usage.get('seconds_used', 'n/a')}s")
print(f"Verfügbare QPU-Zeit: {usage.get('seconds_limit', 'n/a')}s")
```

2. Datensätze vorbereiten

Identische Pipeline wie in den Simulator-Notebooks.

```
In [ ]: # --- Iris ---
iris = load_iris()
X_iris = iris.data[:, [0, 2]]
y_iris = iris.target

X_train_iris, X_test_iris, y_train_iris, y_test_iris = train_test_split(
    X_iris, y_iris, test_size=0.3, random_state=42, stratify=y_iris
)
scaler_iris = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_iris_sc = scaler_iris.fit_transform(X_train_iris)
X_test_iris_sc = scaler_iris.transform(X_test_iris)

# --- Breast Cancer ---
bc = load_breast_cancer()
X_bc = bc.data
y_bc = bc.target

X_train_bc, X_test_bc, y_train_bc, y_test_bc = train_test_split(
    X_bc, y_bc, test_size=0.3, random_state=42, stratify=y_bc
)
pca = PCA(n_components=2, random_state=42)
X_train_bc_pca = pca.fit_transform(X_train_bc)
X_test_bc_pca = pca.transform(X_test_bc)

scaler_bc = MinMaxScaler(feature_range=(0, 2 * np.pi))
X_train_bc_sc = scaler_bc.fit_transform(X_train_bc_pca)
X_test_bc_sc = scaler_bc.transform(X_test_bc_pca)

print(f"Iris - Train: {X_train_iris_sc.shape} Test: {X_test_iris_sc.shape}")
print(f"Breast Cancer - Train: {X_train_bc_sc.shape} Test: {X_test_bc_sc.shape}")
```

3. Pass Manager (Transpilierung)

Circuits müssen vor Hardware-Jobs transpileiert werden.

```
In [ ]: pm = generate_preset_pass_manager(backend=backend, optimization_level=1)
print(f"Pass Manager erstellt für: {backend.name}")
```

4. VQC - Iris (IBM Hardware)

Gleiche Architektur wie Simulator-Experiment.

```
In [ ]: feature_map_iris = zz_feature_map(feature_dimension=2, reps=1)
ansatz_iris = real_amplitudes(num_qubits=2, reps=2)

sampler_hw = Sampler(backend)

vqc_iris_hw = VQC(
    feature_map=feature_map_iris,
    ansatz=ansatz_iris,
    optimizer=SPSA(maxiter=150),
    sampler=sampler_hw,
    pass_manager=pm,
)

print("Starte VQC Iris Training auf IBM Hardware...")
print("⚠ Job wird asynchron ausgeführt - Laufzeit abhängig von Queue")

start = time.time()
vqc_iris_hw.fit(X_train_iris_sc, y_train_iris)
train_time_iris_hw = round(time.time() - start, 4)

print(f"Training abgeschlossen in {train_time_iris_hw}s")
```

4.1 Evaluation & Speichern - Iris

```
In [ ]: start = time.time()
y_pred_iris_hw = vqc_iris_hw.predict(X_test_iris_sc)
infer_time_iris_hw = round(time.time() - start, 4)

acc_iris_hw = accuracy_score(y_test_iris, y_pred_iris_hw)
f1_iris_hw = f1_score(y_test_iris, y_pred_iris_hw, average="weighted")

print(f"Accuracy: {acc_iris_hw:.4f} | F1: {f1_iris_hw:.4f}")
print(f"Training: {train_time_iris_hw}s | Inferenz: {infer_time_iris_hw}s")
print()
print(classification_report(y_test_iris, y_pred_iris_hw, target_names=iris.t

save_result(
    "VQC (RealAmplitudes)",
    "Iris (3 Klassen, 2 Features, hardware)",
    backend.name,
    acc_iris_hw, f1_iris_hw,
    train_time_iris_hw, infer_time_iris_hw,
    Feature_Map="zz_feature_map", Reps=1
```

```
)  
print("Gespeichert.")
```

5. Kontingent prüfen vor Breast Cancer Job

⚠️ Erst prüfen, dann weitermachen.

```
In [ ]: usage = service.usage()  
print(f"Verbrauchte QPU-Zeit: {usage.get('seconds_used', 'n/a')}s")  
print(f"Verfügbare QPU-Zeit: {usage.get('seconds_limit', 'n/a')}s")  
print()  
print("Nur weitermachen wenn noch ausreichend Kontingent vorhanden.")
```

6. VQC - Breast Cancer (IBM Hardware)

```
In [ ]: feature_map_bc = zz_feature_map(feature_dimension=2, reps=1)  
ansatz_bc = real_amplitudes(num_qubits=2, reps=2)  
  
sampler_hw2 = Sampler(backend)  
  
vqc_bc_hw = VQC(  
    feature_map=feature_map_bc,  
    ansatz=ansatz_bc,  
    optimizer=SPSA(maxiter=150),  
    sampler=sampler_hw2,  
    pass_manager=pm,  
)  
  
print("Starte VQC Breast Cancer Training auf IBM Hardware...")  
start = time.time()  
vqc_bc_hw.fit(X_train_bc_sc, y_train_bc)  
train_time_bc_hw = round(time.time() - start, 4)  
  
print(f"Training abgeschlossen in {train_time_bc_hw}s")
```

6.1 Evaluation & Speichern - Breast Cancer

```
In [ ]: start = time.time()  
y_pred_bc_hw = vqc_bc_hw.predict(X_test_bc_sc)  
infer_time_bc_hw = round(time.time() - start, 4)  
  
acc_bc_hw = accuracy_score(y_test_bc, y_pred_bc_hw)  
f1_bc_hw = f1_score(y_test_bc, y_pred_bc_hw, average="binary")  
  
print(f"Accuracy: {acc_bc_hw:.4f} | F1: {f1_bc_hw:.4f}")  
print(f"Training: {train_time_bc_hw}s | Inferenz: {infer_time_bc_hw}s")  
print()  
print(classification_report(y_test_bc, y_pred_bc_hw, target_names=bc.target_  
  
save_result(  
    "VQC (RealAmplitudes)",
```

```

    "Breast Cancer (2 Klassen, 2 Features, hardware)",
    backend.name,
    acc_bc_hw, f1_bc_hw,
    train_time_bc_hw, infer_time_bc_hw,
    Feature_Map="zz_feature_map", Reps=1
)
print("Gespeichert.")

```

7. Optional: Quantum Kernel SVM (reduzierter Datensatz)

⚠ Nur ausführen wenn noch ausreichend Kontingent vorhanden (Zelle 5 prüfen).
 Reduzierter Datensatz (20 Samples) um Kontingent zu schonen.
 In der Arbeit explizit als "Hardware-Experiment mit reduziertem Datensatz" kennzeichnen.

```

In [ ]: # Reduzierter Datensatz - nur 20 Trainingssamples
X_train_iris_small = X_train_iris_sc[:20]
y_train_iris_small = y_train_iris[:20]

print(f"Reduzierter Datensatz: {X_train_iris_small.shape}")
print(f"Klassenverteilung: {np.bincount(y_train_iris_small)}")

feature_map_qk = zz_feature_map(feature_dimension=2, reps=2)
kernel = FidelityQuantumKernel(feature_map=feature_map_qk)

start = time.time()
svc_q = SVC(kernel=kernel.evaluate)
svc_q.fit(X_train_iris_small, y_train_iris_small)
train_time_qk_hw = round(time.time() - start, 4)

start = time.time()
y_pred_qk_hw = svc_q.predict(X_test_iris_sc)
infer_time_qk_hw = round(time.time() - start, 4)

acc_qk_hw = accuracy_score(y_test_iris, y_pred_qk_hw)
f1_qk_hw = f1_score(y_test_iris, y_pred_qk_hw, average="weighted")

print(f"Accuracy: {acc_qk_hw:.4f} | F1: {f1_qk_hw:.4f}")
print(classification_report(y_test_iris, y_pred_qk_hw, target_names=iris.target_names))

save_result(
    "Quantum Kernel SVM (20 Samples)",
    "Iris (3 Klassen, 2 Features, hardware, reduziert)",
    backend.name,
    acc_qk_hw, f1_qk_hw,
    train_time_qk_hw, infer_time_qk_hw,
    Feature_Map="zz_feature_map", Reps=2
)
print("Gespeichert.")

```

8. Gesamtvergleich: Ideal → Noise → Hardware

```
In [ ]: df = pd.read_csv("Ergebnisse/ergebnisse.csv")

# Alle VQC Einträge über alle Backends
vqc_rows = df[df["Modell"].str.contains("VQC")].copy()
vqc_rows = vqc_rows[["Modell", "Datensatz", "Backend", "Accuracy", "F1", "Tr

print("VQC - Vergleich über alle Backends:")
print(vqc_rows.to_string(index=False))
```